Conceptual Architecture of WezTerm

CISC 322/326

Assignment 1

October 2026

Group 3: DeadBool

Christian Aitas: 23JPR3@queensu.ca
Calvin Dang: Calvin.dang124@gmail.com
Rocco Destefano: rocco.destefano@queensu.ca
Darby Duggan: 23HB83@queensu.ca
Addie Kucey: 23FRM2@queensu.ca
Victor Magnuszewski: 23HWX@queensu.ca
Mandatory Virtual AI Teammates:
ChatGPT-5.6 Sol
DeepSeek-V4.1-Flash

Table of Contents

1 Abstract **2**

2 Introduction and Overview **2**

3 System Functionality and Architecture **3**

4 Team Development of WezTerm **6**

5 Conceptual Concurrency Overview **7**
5.1 Core Conflict 7
5.2 Concurrency Sectors 7
5.3 Bridges 8
5.4 Concurrency Summary 8

6 System Evolution **9**

7 Control and Data Flow **9**

8 Use Cases **10**
8.1 Opening a New Local Tab: Sequence Diagram 10
8.2 Opening a remote connection 12

9 External Interfaces **12**

10 Data Dictionary **13**

11 Naming Conventions **13**

12 Conclusions **13**

13 Lessons Learned **14**

14 AI Collaboration Report **14**

15 References **16**

1 Abstract

This report presents the conceptual architecture of WezTerm, a GPU-accelerated, cross-platform terminal emulator and multiplexer written in Rust. We describe the system as four main components: multiplexer and session management, terminal emulation, GUI and rendering, and Lua-based configuration and automation. We then look at how they work together. The architecture combines implicit invocation, client-server, and pipe-and-filter styles, which lets WezTerm handle both local and remote sessions while keeping terminal processing separate from rendering.

Beyond the architecture itself, we look at how a team could divide the work of building WezTerm and why clear interfaces matter, especially for a cross-platform, open-source project. We also cover WezTerm's concurrency model, which keeps three kinds of work apart so that none of them slows down the others: the main interface, network communication with remote sessions, and waiting on the shell for output. Two bridges, the promise module and the spawn queue, let these parts pass tasks to each other safely. The report traces WezTerm's evolution from a simple terminal emulator in 2018 to a larger system with multiplexing, remote domains, and Lua configuration. It also follows the control and data flow from a keypress through to the rendered output, and walks through the "open a new local tab" use case in a sequence diagram.

Overall, we found that WezTerm's strengths come from its clearly separated responsibilities, event-driven design, and careful handling of concurrency. Together these keep a feature-rich system maintainable, extensible, and responsive.

2 Introduction and Overview

Working in a terminal usually means executing commands and studying their output. However, programmers often have to perform different tasks at once, launch and debug applications, and switch between tasks. Such a situation puts a great burden on the terminal since all processes take place in one window or, although separated, are open locally on the computer. However, there are alternative terminal applications that help perform tasks more efficiently by implementing various improvements. In particular, the WezTerm program offers to manage multiple processes at once and work with remote machines more conveniently [1].

The first advantage of using WezTerm is managing previous outputs. When working with a terminal, applications often print much information that is necessary to consider for debugging or simply finding an error. In this case, the scrollback function helps, which is implemented in the program. Using this function, one can search through previous outputs and scrolls them up. At the same time, the amount of information saved in the scrollback area is user-specified, which is convenient for further use. Another feature of such programs as WezTerm is working with hyperlinks. In particular, hyperlinks in the output of applications can be highlighted and opened directly in the browser. Moreover, it becomes possible to create links with specific text so that clicking on them takes the user to the desired web page [1].

The ability to work with multiple terminals is another benefit of using WezTerm, which is

designed as a multiplexer. Multiplexing programs differ in that in them, the user can control domains, which can be separated tabs or entire computers connected via the network. In particular, while local domains are used by default, WezTerm offers the user many other options, such as an SSH connection or a secure connection over TLS. In addition, for local connections, there is an option for creating Unix sockets [2] .

From a user's point of view, the described features highlight the advantages of the WezTerm program. At the same time, the implementation of all these features requires a huge number of various actions to be executed at once. For example, changing or creating new windows implies changing the current state of the application, storing data about the current session, and transmitting this information to the user. However, when working with remote domains, the interaction of multiplexer and computer software becomes even more complicated. Thus, the study of the architecture of the newly created program will allow an understanding of the principles of operation of such structures. The aim of this paper is to analyze the design of the WezTerm application, thereby demonstrating the importance of the separation of responsibilities and interaction between program components.

3 System Functionality and Architecture

System Functionality

WezTerm is a GPU-accelerated cross-platform terminal emulator and terminal multiplexer programmed in Rust. It provides a graphical environment in which users can run command-line programs while organizing their terminal sessions into panes, tabs, windows, and workspaces. WezTerm runs on Linux, MacOS, and Windows and supports both local and remote terminal sessions. It has multiple features that enhance the user experience, including GPU-accelerated rendering, extensive customization options, cross-platform compatibility, terminal multiplexing, and support for multiple tabs and panes. [1] One important distinction is that WezTerm itself is not a shell such as Bash. Instead, it creates a terminal environment in which those programs run. As well, WezTerm has an inbuilt multiplexer functionality. A multiplexer organizes running programs into a hierarchy consisting primarily of panes, tabs, windows, and workspaces. On Unix-like systems, a terminal emulator creates a pseudo-terminal (PTY), launches a shell or another program inside it, interprets the program's output for display, and translates keyboard and mouse input into data that can be sent back to the running program. [3]

Main System Components

The main system compnonets would be the following: Multiplexer and Session Management, Terminal Emulation, Graphical User Interface and Rendering, and Configuration and Automation.

Multiplexer and Session Management.

Multiplexing is one of the main components WezTerm. The multiplexer is responsible for the management and organization of the collection of terminal sessions available to the user. In other terms, the multiplexer manages running programs in terms of panes, tabs, windows, and workspaces without the need of a GUI. [2]

Terminal Emulation

The terminal emulator is the component that handles the output produced by command-

line programs. When a local shell or application is running, it communicates with WezTerm through a pseudo-terminal (PTY). The PTY acts like a connection between the program and the terminal. The program sends its output through the PTY and then WezTerm receives that data. The terminal emulator then interprets the information it receives and figures out how it should appear on the screen. This includes normal text and can include escape sequences. Escape Sequences are special instructions that can tell WezTerm to do things such as move the cursor or change visuals. In this way, the terminal emulator is responsible for turning the raw output from a command-line program into the user's terminal display.

Graphical User Interface and Rendering

The Graphical User Interface and rendering component is responsible for displaying all visual elements for the user. It shows the terminal windows, tabs, panes, text, colour, and other visual elements. The GUI also handles user interactions such as keyboard input, mouse input, scrolling, and switching between different terminal sessions. The rendering part takes the information maintained by the terminal emulator and converts it into what is actually shown on the screen. As well, text rendering also involves font shaping, which is taking in multiple text characters and transforming them into a single font glyph to aid in readability. WezTerm has the option to use the GPU on the computer to accelerate the render of the terminals.

Configuration and Automation

The Configuration and Automation component is responsible for allowing the user to customize the visuals and behaviours of WezTerm. WezTerm uses a light weight scripting language LUA to do all the customizations. [4] WezTerm uses a Lua configuration file in which the users can change settings such as fonts, colours, key bindings, window behaviour, and other terminal options. It is also possible to set more advanced behaviour using functions and events. The configuration can affect multiple parts of WezTerm which include the GUI, multiplexer, and terminal sessions. [5]

Interaction Between Components

The main components of WezTerm interact through two paths, the control path and the terminal data path. The control path handles events from the user or the system through the GUI. Depending on the event, WezTerm sends instructions to the multiplexer, such as creating a new pane, switching workspaces, or starting a process. The multiplexer then manages these requests and communicates with the appropriate terminal process through its domain. Any changes to the session are then reflected in the GUI. The terminal data path handles the input and output between the user and the running application. When a user types a command, the input travels from the GUI through the active pane and domain to the PTY or remote connection. The application then sends its output back through the same connection. WezTerm processes this output, including any escape sequences, and updates the terminal state. The rendering component then takes this information and displays it in the GUI. By separating terminal processing from rendering, WezTerm can change how information is displayed without affecting how it communicates with the running application. Configuration and automation also play a role in how these components interact. WezTerm uses Lua configuration files to define settings and behaviour, while event callbacks allow certain actions to be triggered automatically. For example, startup and window events can trigger actions such as creating new panes or managing windows. Rather than being part of a

single processing path, the configuration component interacts with several other components to control how WezTerm behaves.

Architectures

WezTerm uses a combination of different architecture styles, such as layered, implicit invocation, and client-server, depending on the component and its responsibilities. Overall, WezTerm can be best described as a loosely layered architecture coordinated by the multiplexer. The GUI and CLI layers provide user and command entry points and then the multiplexer layer coordinates the domains. Afterwards, domain adapters connect panes to local programs.

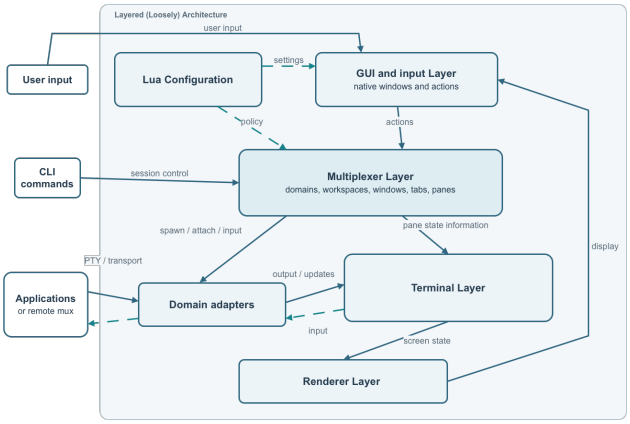


Figure 1: Architecture

Another architecture style used in WezTerm is the implicit invocation. When an event occurs, such as a window being opened or a user performing an action, WezTerm can call the registered event handlers to respond. This allows different parts of the system to react to events without needing to directly communicate with each other. It also makes the system more flexible, as additional behaviours can be introduced through Lua scripts. [6] Another style that WezTerm uses is the client-server architecture as shown through with remote terminal sessions. For example, when connecting through an SSH multiplexing domain, the local WezTerm client can communicate with a multiplexer daemon running on a remote machine. The remote multiplexer manages the terminal sessions, while the local GUI allows the user to interact with them. This separation allows terminal sessions to be managed remotely without requiring the GUI and the running applications to be on the same machine.

Another style that can be can found is the is pipe-and-filter. When the user enters a command, the input passes through different components before reaching the running application. The output then passes through the terminal emulator, which processes the data and updates the terminal state, before being displayed by the rendering component. Separating the GUI from the multiplexer makes it possible to manage both local and remote sessions, while separating terminal emulation from rendering allows the display system to be changed without affecting terminal processing.

To develop WezTerm's conceptual architecture, we reviewed the official documentation and examined two use cases: opening a local tab and connecting to a remote session. These helped us identify the main components and dependencies shown in Figure 1 while keeping the architecture at a conceptual level.

An alternative to WezTerm's loosely layered architecture would be a monolithic architecture, which could simplify communication but make maintenance harder. A strictly layered architecture could improve organization but limit flexibility when handling events and remote connections. WezTerm's loosely layered approach balances flexibility and separation of responsibilities, making it easier to manage sessions and add new features.

4 Team Development of WezTerm

Depending on a software system's architecture layout, a software team's collaborative structure can be entirely different. The architecture's responsibilities should be divided among participating developers involved in the project. The system is divided into smaller subsystems, which can be assigned to a developer or team. These subsystems have interfaces to the adjacent subsystems, allowing the developers to do their own parts independently. WezTerm is cross-platform, meaning that it functions on multiple operating systems such as Mac, Linux, and Windows. The software being cross-platform can cause some difficulty if all the components aren't adjusted for the proper operating system. The concepts of the components are universal, but how they are implemented is different per operating system. The division of responsibilities among participating developers should relate to the developers' skills.

The table below summarizes the different roles, their responsibilities, and the specialized knowledge needed. Each role relates to one of the main features of WezTerm or to the flow of data between the user and the shell.

Role	**Responsibility**	**Specialized Knowledge Needed**
Input handling	Captures keyboard and mouse input and passes it on	Windowing and OS event systems
PTY / shell interface	Connects WezTerm to the shell through the OS	Per-platform OS internals (Linux, macOS, Windows)
Output processing	Reads and interprets the shell's output	Terminal standards and parsing
Rendering	Draws text using the GPU	GPU programming and text rendering, which differ per platform
Lua configuration	Loads and applies the user's Lua configuration	Lua scripting and API design
Multiplexer	Manages the windows and tabs that make up the terminal session, runs several terminals at once, and keeps sessions alive so a user can close the terminal and go back to it later	Concurrency, process and session management, networking for remote sessions
Technical lead	Defines interfaces and integrates all parts	Whole-system view

Table 1: Roles and responsibilities in WezTerm

All these roles require specialized knowledge. When dividing the work among the team, it is important to assign the work according to the developers' skillsets. When working on the multiplexer feature, for example, developers should manage multiple windows and tabs to run the terminal session. For this feature, it is important that the developer has concurrency skills. For WezTerm, this looks like running several terminals at once, possibly running different shells. Some roles may require a small team to ensure that their task runs on multiple different operating systems. Even with the correct specialists for each role, the roles can't be built on their own. Each role depends on the other since the output of one stage in the program goes to the next stage. For example, the shell interface stage relies on the user input. Because all these roles depend on each other, the interfaces between them determine how indepen-

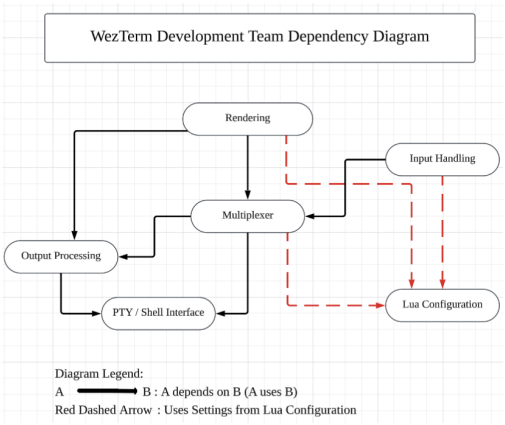


Figure 2: WezTerm Development Team Dependency Diagram

dently each developer can work. Figure 2 describes how the different roles depend on one another.

When integrating these parts, the technical lead should phase the development so that early versions of the full system or significant parts of the system are integrated and made to work. This allows for identification of integration issues early on. This requires having everyone build a minimal version of their component, that can then be tested when put all together. This will be the base and then features such as Lua configuration and the multiplexer can be added in. Cross-cutting concerns such as security, error handling, and logging will affect multiple interfaces if not handled delicately, so it is important to start with a simple system to get those pieces working and build up. When discussing the division of responsibilities among participating developers for WezTerm, it is important to note that WezTerm is an open-source project. So, its development is different from that of a typical corporate project. Contributors to the project will often be volunteers who decide themselves what to work on, not developers who are assigned roles. There is a small group of developers who maintain the project and approve and review contributions. Clear interfaces are still important as they let outside contributors work on a single area of the program, such as the graphics, without changing the whole program.

5 Conceptual Concurrency Overview

5.1 Core Conflict

In a GUI application, there is usually a special GUI processing loop that must run on the main thread. It is usually impossible to run a Tokio event loop in that context, as the GUI event loop needs to own the main thread. Asynchronous runtimes want to run their own event loops, but the GUI system already has a proprietary one. Wezterm's concurrency model exists to reconcile three incompatible execution contexts. The main GUI event loop, blocking pseudo terminal reads and asynchronous network I/O. I will be referring to these three different contexts as "sectors". [7, 8]

5.2 Concurrency Sectors

At the conceptual level, WezTerm's execution is divided into three separate sectors, each with different responsibilities and concurrency primitives. The GUI sector is the main thread of the application. It owns the operating system windowing event loop, processes user inputs (keyboard and mouse) and drives the rendering pipeline. The main constraint of this sector is that OS windowing APIs are not thread safe. Certain display handles can only be accessed form the thread that initialized it. Any other thread that attempts to interact with the window will panic. For this reason, all window operations such as creating panes, resizing tabs and updating the currently rendered display must be run through this thread. The GUI sector is the only execution context that is allowed to touch the windowing subsystem. [7, 9]

The asynchronous sector is a secondary execution context. It exists primarily to handle network I/O for the multiplexer subsystem. WezTerm's multiplexer supports remote domains over SSH, Unix domain sockets and TLS (secure remote domain) connections. Each of these

waits on the socket to be ready without blocking a thread. The asynchronous sector lets WezTerms multiplexor to manage many simultaneous connections efficiently. The system will park futures when no data is available and resume them when the required socket becomes readable. This sector cannot run on the main thread because the GUI already owns that event loop. The two loops can't run on the same thread at the same time. WezTerm isolates these two loops so that it can perform its own task scheduling without interfering with the GUI. [10, 11]

Finally, the blocking sector exists for operations that inherently wait for long periods.The most apparent example is reading from the pseudo terminal (abbreviated PTY) that runs the user's shell. The `pty_reader.read()` function call is a blocking operation. The function will not return until the shell produces an output. If this call were made on the GUI thread, the entire terminal would freeze whenever the shell wasn't giving an output. If it were made on the Asynchronous Tokio runtime thread, it would stall the async executor and prevent other networking tasks from progressing. Therefore, the blocking subsystem consists of dedicated background threads that block harmlessly while keeping the GUI and asynchronous sectors responsive. [12]

5.3 Bridges

The big problem with isolating these three sectors is that they can't call each other's functions directly, as they exist in different execution contexts. To connect these gaps, WezTerm uses two conceptual bridges to enable subsystem communication. The `promise` module is the main bridge. It provides a function, `spawn_into_main_thread(future)`, which wraps a future and schedules it to be run on the GUI thread. This is how the asynchronous sector safely interacts with main thread only resources. It also contains `tokio_spawn(future)` for spawning work into the Asynchronous threads, and `spawn_into_new_thread(f)` for offloading blocking work to the blocking threads. The module also allows the GUI application to schedule normal and low priority futures on the main event thread. [13]

Finally, the `SPAWN_QUEUE` is the concrete mechanism that makes these bridges work. It is a queue that holds tasks scheduled by the `promise` module mentioned above. Each platform's GUI event loop is configured to check this queue. When a new item appears in the queue, the GUI sector wakes up to process it, ensuring tasks sent from the Asynchronous sector are executed safely and within the correct context. [13, 14]

5.4 Concurrency Summary

This hybrid model exists because the three sectors have incompatible concurrency requirements. The GUI sector cannot be replaced by another runtime because the OS systems need main thread ownership. Blocking calls would slow the asynchronous sector heavily. The bridges that are the `promise` module and `SPAWN_QUEUE` allow each sector to operate in its own context while still enabling cross subsystem communication and coordination. The result is a concurrency model that works around the constraints of each subsystem while providing a single, responsive user experience. [7, 12, 13]

6 System Evolution

WezTerm began as a terminal emulator created by Wez Furlong around 2018, originally designed to provide users with a reliable environment for interacting with a shell. As the system developed, it introduced features such as multiple tabs, remote connections, serial device support, and image display. By 2020, WezTerm continued to improve its usability through better text handling, resizing, session management, and customization options [15].

One of the most important developments was the introduction of multiplexing, which allowed users to organize and manage multiple terminal sessions through windows, tabs, and domains. This improved how local and remote sessions were handled within the same application [2]. Remote access also became a major part of WezTerm's evolution, allowing users to connect to and manage sessions running on other computers [1]. Another significant improvement was the introduction of Lua-based configuration, which gave users greater control over the terminal's appearance and behavior as new features were added.

As WezTerm became more complex, its architecture evolved to separate responsibilities between different components, including the user interface, session management, and remote connections. This allowed the system to support additional features without keeping all its functionality tightly connected. WezTerm also improved its communication with the shell by recognizing information such as working directories, prompts, and command output, making it easier to navigate between commands and open new tabs or panes. Over time, features such as workspaces, stronger remote support, and improved session management further expanded its capabilities. Overall, WezTerm evolved from a basic terminal emulator into a flexible system capable of managing multiple local and remote terminal environments while providing users with greater control and customization [15, 1].

7 Control and Data Flow

At the conceptual level, WezTerm's control and data flow can be demonstrated through the interaction of many major parts such as the GUI, the multiplexer, terminal panes, pseudoterminals, the programs running inside those terminals, and the terminal rendering system. There are two different types of information that can move between these components. Control flow represents actions that change the state of WezTerm itself. Actions such as creating a pane, switching tabs, or opening a new terminal are all affected through this flow. Data flow represents the actual terminal information travelling through the system. This is primarily demonstrated by keyboard input travelling toward a running application and application output travelling back toward the user.

The majority of interactive control flow begins when the GUI receives a keyboard or mouse event from the user. For keyboard input, WezTerm will process events and determine whether they match a pre-configured key assignment. A KeyAssignment represents an operation that affects the state of a window, tab, or pane. If the input matches an assignment, it is treated as a control instruction rather than being sent directly into the program running in the terminal. For example, the action SplitPane causes WezTerm to split the active pane and create a new command line inside the new pane. These control operations are closely associated with WezTerm's multiplexer. The multiplexer organizes running programs into

panes, tabs, windows, and workspaces. When a user inputs an operation that affects the terminal structure, the multiplexer maintains the logical state of those terminal sessions.

WezTerm notes that the multiplexer can operate without being directly connected to a GUI. As a result, the GUI should not be seen as the component that directly owns every terminal process. It acts as the interface through which the user sends requests, while the multiplexer manages the corresponding terminal resources. Not all control flow originates from keyboard inputs. Events such as changes to a window size and tab selection can cause WezTerm to update its state. These events still follow the same architectural principles. The global control flow in WezTerm can be seen as event driven. The user or system events enter through the GUI, WezTerm interprets those events as actions, and the corresponding request is directed toward the subsystem responsible for that ressource. The most common resource being the multiplexer, responsible for coordinating terminal session structure. The GUI will then present the updated result to the user [2, 10].

Data flow occurs when a keyboard event is intended for the application running inside the terminal rather than for WezTerm itself. If a keyboard input does not trigger a WezTerm action, it can be encoded as terminal input sent toward the active pane. A pane represents an individual terminal session and maintains the connection between WezTerm and its associated program. The connection between a pane and a shell is provided by a pseudoterminal (PTY). A PTY provides an input/output interface that allows a terminal emulator like WezTerm to communicate with programs such as Bash, PowerShell, or other command-line applications. If a user enters a command such as ls, the terminal input is sent through the pane and PTY to the program currently using the terminal. The program will then receive the input as if it were communicating with a traditional hardware terminal.

The output of the running application follows the same principles in the opposite direction. Once a shell processes a user's input, it sends the output back through the PTY to WezTerm. WezTerm will interpret the output and update the terminal state. Once the terminal state has been updated, the information is passed to WezTerm's rendering system, where the current terminal state will be displayed graphically to the user. The result is a cycle in which user input travels to the running application and application output travels back to the user.

8 Use Cases

8.1 Opening a New Local Tab: Sequence Diagram

The OpenNewLocalTab sequence diagram illustrates how WezTerm's components interact when a user opens a new local terminal tab within an existing window. Note that this use case assumes that there is an existing window, a local domain, and successful creation. This use case focuses on the interactions between the GUI, Multiplexer, Local Domain, PTY / Process Subsystem, and Terminal Emulation while abstracting asynchronous scheduling.

The sequence begins when the user requests a new tab through the GUI, which then forwards the request to the Multiplexer, where it then identifies the window where the tab should be created. The Multiplexer then resolves the target domain, terminal size, and working directory, which establishes the context needed for the new terminal session. This

data is then passed on to the Local Domain, which is responsible for launching the session on the user's machine.

The Local Domain then prepares the shell command and requests a pseudo-terminal, or PTY, from the PTY / Process Subsystem, which is responsible for providing the communication channel between WezTerm and the shell. Once a PTY has been allocated, the subsystem starts the shell process.

Next, the Local Domain initializes the Terminal Emulation component with the terminal size and a writer connected to the PTY. The emulator then keeps track of the terminal's state, which includes what the terminal currently contains and how it should appear. This is needed to interpret program output, including text, cursor positions, and display attributes. The Local Domain then creates a pane, which links the terminal instance, PTY, and shell process, and creates a tab containing that pane (these operations are grouped into a single conceptual step in the diagram).

The Local Domain registers the new pane and tab with the Multiplexer and links the tab to the existing window. After registration, the Local Domain returns the created tab and then the Multiplexer selects it as the active tab and returns the relevant information back to the GUI. Finally, the GUI then renders the active tab, which completes the user's request.

This use case demonstrates each of the different responsibilities divided amongst each component within WezTerm. The GUI handles the user requests and presentation, the Multiplexer coordinates session organization, the Local Domain manages local session creation, the PTY / Process Subsystem is responsible for the terminal connection and shell process, and the Terminal Emulation oversees the terminal's state.

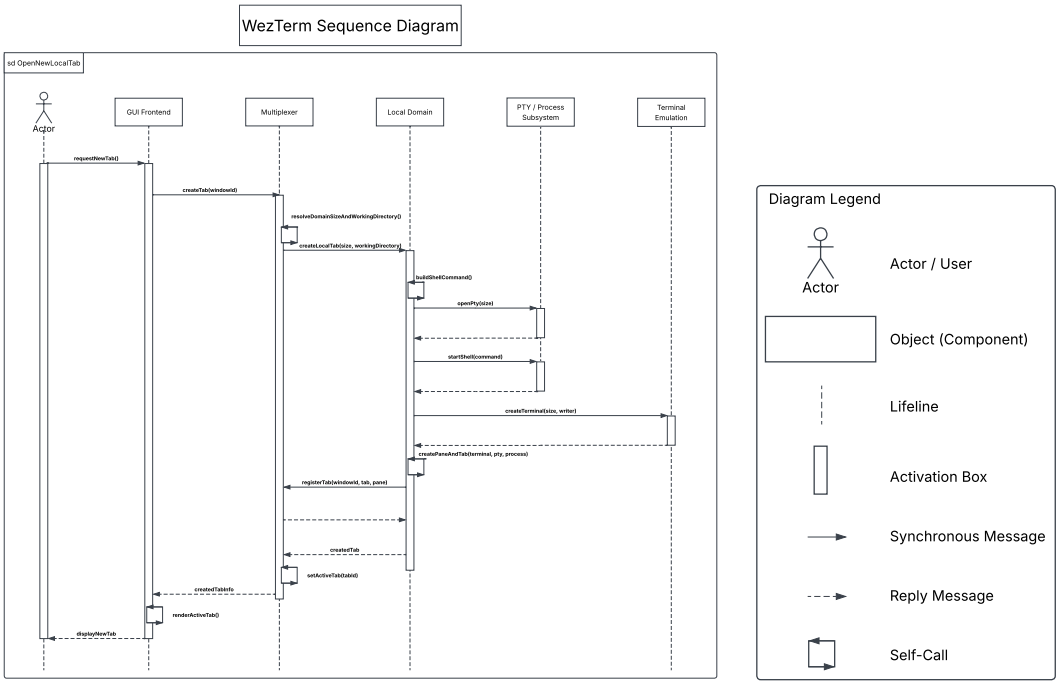


Figure 3: Sequence diagram for opening a new local tab in WezTerm, with notation legend.

Note: This diagram assumes that there is an existing window, a local domain, and successful creation.

Messages describe conceptual component interactions, not exact method signatures. Asynchronous scheduling is abstracted.

8.2 Opening a remote connection

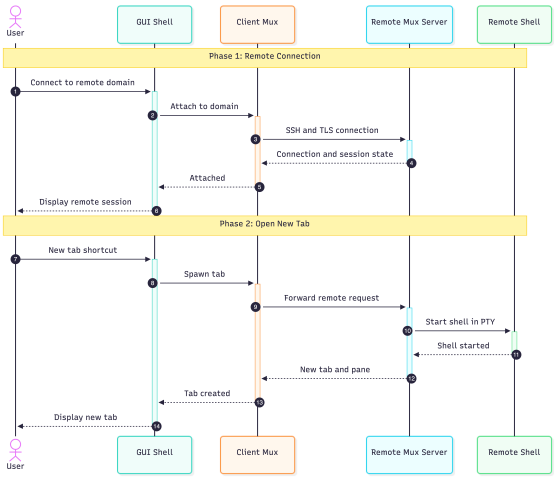


Figure 4: Sequence diagram for remote connection

In the first phase, the user executes wezterm connect to access a configured remote domain. The GUI Shell requests attachment through the Client Mux, which bootstraps the Remote Mux Server using SSH (wezterm-client/src/client.rs). The Client Mux then establishes a TLS-encrypted connection and retrieves the existing session state, which is forwarded to the GUI Shell. In the second phase, the user opens a new terminal tab. The GUI Shell forwards the request through the Client Mux to the Remote Mux Server over TLS. The server creates a PTY,launches a shell process, and returns the new tab and pane information to the GUI Shell for display.

9 External Interfaces

External Interfaces. The most fundamental interface is the pseudo-terminal (PTY) connection to running programs. WezTerms spawns a shell or application and exchanges two types of information, outbound and inbound. Outbound information consists of the user's keyboard inputs, controls such as window resize notifications, and session control commands (eg., interrupt signals). Inbound information is made up of program outputs, plain text and ANSI escape sequences that describe desired formatting, colours and cursor positioning. Western parses and renders the necessary elements.

Graphical user interface The GUI in itself is also an interface, receiving mouse events, and keyboard events from the OS. It also supports input method editors for complex texts. The interface is how the user interacts with the terminal directly.

Multiplexing and Remote domains. WezTerm can connect to remote domains, via SSH, domain sockets or TLS. Outbound: Multiplexer control commands (creating tabs and panes, splitting windows), and session management Requests. Inbound: Terminal output from these remote processes and status updates.

Configuration and local data. WezTerm reads configuration from a Lua file. This file provides the user's config preferences. Colour schemes, key bindings, font settings, domain definitions, custom keyboard shortcuts and event handlers. It is a file-based interface that determines the system's behaviour at startup and upon reload. Local data also includes scrollback history, command history and SSH configurations.

10 Data Dictionary

GUI and Rendering: Handles input and displays terminal output.
Multiplexer: Manages terminal sessions that are split into panes, tabs, and windows. Allows for multiple terminal sessions at once.
Domain: A set of windows and tabs, for local or remote sessions.
Pane: A single terminal session.
PTY: Provides communication between WezTerm and a shell.
Terminal Emulator: Interprets program output and updates the terminal state.
Terminal State: What the terminal is currently showing.
Lua Configuration: Controls user settings such as the colours, the fonts, and different program behaviour. Coded in Lua.
Control Flow: Input and output to the program. Changes to WezTerm.

11 Naming Conventions

GUI (Graphical User Interface): Handles input (from keyboard or mouse) and displays terminal output.
PTY (Pseudo-Terminal): Provides communication between WezTerm and a shell.
SSH: Used to create connections to remote terminal sessions.
TLS: Used to secure connections to the remote domains.

12 Conclusions

On the surface WezTerm looks like a terminal emulator but there's a lot going on underneath. It handles emulation, multiplexing, GPU rendering and configuration. It has to do all of that across operating systems. Each of these parts has its job and the system only works because they talk to each other. A keypress has to get to the shell and whatever the shell prints back to show up on screen The clearest thing about the design is how cleanly the responsibilities are split. The GUI handles input and drawing. The multiplexer keeps track of panes, tabs, windows and workspaces. The emulator makes sense of what programs output. The Lua configuration lets users see how everything looks and behaves. Because of this split it's much easier to follow what the system is doing. It's easier to repair it when something breaks. It's easier to add to it. Following the control and data flow from a keypress to the shell and back to the rendered output showed how these pieces fit together in practice.

Concurrency matters as much. WezTerm keeps work, asynchronous networking and blocking operations in split execution contexts. So a slow task doesn't stop the interface. Messages passed between those contexts rescue sessions running smoothly. It also affects how a team could work on the project. Since people can focus on one subsystem they can coordinate with others through defined interfaces. WezTerm's history shows why this matters. It started as an emulator. Then it picked up multiplexing. Then remote connections. Then GPU-accelerated rendering. Then deep customization. Every addition made the system more complicated. That made clear boundaries between components more necessary.

Overall WezTerm is an example of how mixing several architectural approaches can bring both speed and flexibility. Its separated responsibilities, event-driven flow and concurrency model let the components work together without losing their roles. Looking at these choices shows how a complex piece of software can save growing. It can stay maintainable. It can stay extensible. It can stay responsive, for the people using it.

13 Lessons Learned

One of the main lessons our group learned after completing A1 was the importance of proper communication. We were able to share ideas and collaborate online and in person. Although every group member communicated clearly and effectively, there is always room for improvement. For upcoming assignments, we plan to communicate more consistently and divide our responsibilities earlier on in the process. Our group also had difficulty including all of our information within the 17-page limit. In the future, we will organize our work more efficiently to avoid excess content.

Another lesson we learned was the importance of keeping our work at a conceptual level. When researching WezTerm, we found that some information was too technical for what the assignment required. We had to decide which components and interactions were most important to include without going into unnecessary detail. We also learned how important it is to keep our diagrams consistent with the written explanations so that the overall architecture is easier to understand.

14 AI Collaboration Report

AI Member Profile and Selection Process
Throughout the completion of A1, our group used ChatGPT-5.6 Sol along with DeepSeek-V4.1-Flash, as our virtual AI teammates. ChatGPT was selected primarily because this assignment required us to work with a large amount of technical information. We had to be able to understand unfamiliar concepts and communicate each concept effectively in the final report. It was useful at explaining relationships between major WezTerm components as well as defining difficult terminology. ChatGPT was a lot less restrictive than other LLMs as our group used a premium version. DeepSeek was primarily used as an additional analysis tool to bring a second perspective when examining WezTerm's architecture. Our group was able to compare responses rather than relying on the interpretation of a single system. If ChatGPT and DeepSeek produced different explanations of a topic, it gave us a reason to further investigate using credible sources. Neither AI was treated as a reliable source. Their responses were used to guide our research and understanding, while the claims included in the final report were checked against the official WezTerm documentation, as well as other primary sources.
Tasks Assigned to AI Teammates

- Both ChatGPT and DeepSeek were used to provide clear descriptions and explanations of WezTerm and architecture concepts that some group members may have been

unfamiliar with. This was appropriate for AI as it can quickly simplify technical terminology and provide a starting point for further research.

- DeepSeek helped locate areas of the WezTerm documentation where useful information was harder to find. This was helpful as it reduced the amount of time spent researching while still allowing group members to verify the information themselves using original sources.
- We used ChatGPT to determine whether our descriptions remained at the conceptual architecture level required for A1. This was appropriate for AI as we could compare our drafts against the project guidelines to make sure we were on the right path.
- Both ChatGPT and DeepSeek helped with the structure and formatting of our report written in LaTeX. They also helped with grammar suggestions. This was appropriate as all the technical content was still researched and verified, the AIs helped improve how the information was communicated.

Interaction Protocol and Propmting Strategy

The entire group was able to use AI whilst working on A1. We made sure to keep in mind the quality of our prompts in order to receive high quality output catered to our needs. An example prompt is one we used for a draft review: "Review the following paragraph about WezTerm's control flow. Check whether it clearly explains how control passes between major components and whether it stays at a conceptual architecture level required for A1. Identify any missing points and suggest concise improvements." Prompts were to be as detailed as possible. Our group tried not to use single sentence prompts as much as we could. Context such as the assignment guidelines and primary sources were provided as well.

Validation and Quality Control Procedures

AI generated content was not automatically accepted into the final report as AI can provide inaccurate statements. Any explanations or information provided by ChatGPT or DeepSeek were cross-referenced with primary sources such as the official WezTerm documentation. After writing our report, each member of the group proofread the material to ensure that our work was clear and correct.

Quantitative Contribution and Final Reflection

Our group estimated that our virtual AI teammates contributed to around 20% of the final A1 deliverable. Their largest contribution was with research and explanation, where they helped group members understand unfamiliar concepts and identified areas that needed further attention. They also contributed to the editing and restructuring of our final report as well as help with the initial brainstorming and interpretations of the assignment requirements.

Our AI teammates generally reduced the overall amount of time needed to complete A1. We were able to acquire information quickly and clarify any issues we had relatively easily. At the same time, using ChatGPT and DeepSeek introduced additional work. AI output needed to be fact checked against primary sources. Some responses included more implementation detail that was not appropriate for this part of the project. Our group tried our best to use AI as effectively as possible. This included providing it proper context and inputting higher quality prompts. There were no major challenges or disagreements regarding AI. The main lesson our group learned from this process was that AI was most effective when treated as a collaborative research and review tool rather than as a replacement for the group's work. In the future, we will continue to use AI responsibly by using proper prompts and cross-examining any useful information it provides.

15 References

[1] W. Furlong, "Wezterm features," https://wezterm.org/features.html, accessed: October 7, 2026.

[2] ——, "Wezterm - multiplexing," https://wezterm.org/multiplexing.html, accessed: October 7, 2026.

[3] ——, "Wezterm - what is a terminal?" https://wezterm.org/what-is-a-terminal.html?h=terminal, accessed: October 7, 2026.

[4] Lua.org, "About lua," https://www.lua.org/about.html, accessed: Oct. 7, 2026.

[5] WezTerm, "Configuration reference," https://wezterm.org/config/lua/config/index.html, accessed: Oct. 7, 2026.

[6] ——, "Event callback," https://wezterm.org/config/lua/wezterm/on.html, accessed: Oct. 7, 2026.

[7] WezTerm Project, "WezTerm: format-window-title," https://wezterm.org/config/lua/window-events/format-window-title.html, accessed: October 7, 2026.

[8] ——, "WezTerm: wezterm.gui module," https://wezterm.org/config/lua/wezterm.gui/index.html, accessed: October 7, 2026.

[9] ——, "WezTerm: gui-startup," https://wezterm.org/config/lua/gui-events/gui-startup.html, accessed: October 7, 2026.

[10] ——, "WezTerm: wezterm.mux module," https://wezterm.org/config/lua/wezterm.mux/index.html, accessed: October 7, 2026.

[11] ——, "WezTerm: default_domain," https://wezterm.org/config/lua/config/default_domain.html, accessed: October 7, 2026.

[12] ——, "WezTerm source code: mux/src/lib.rs," https://github.com/wezterm/wezterm/blob/main/mux/src/lib.rs, accessed: October 7, 2026.

[13] ——, "WezTerm source code," https://github.com/wezterm/wezterm, accessed: October 7, 2026.

[14] ——, "WezTerm source code: window/src/os/wayland/window.rs," https://github.com/wezterm/wezterm/blob/main/window/src/os/wayland/window.rs, accessed: October 7, 2026.

[15] W. Furlong, "Wezterm," https://wezterm.org/, accessed: October 7, 2026.